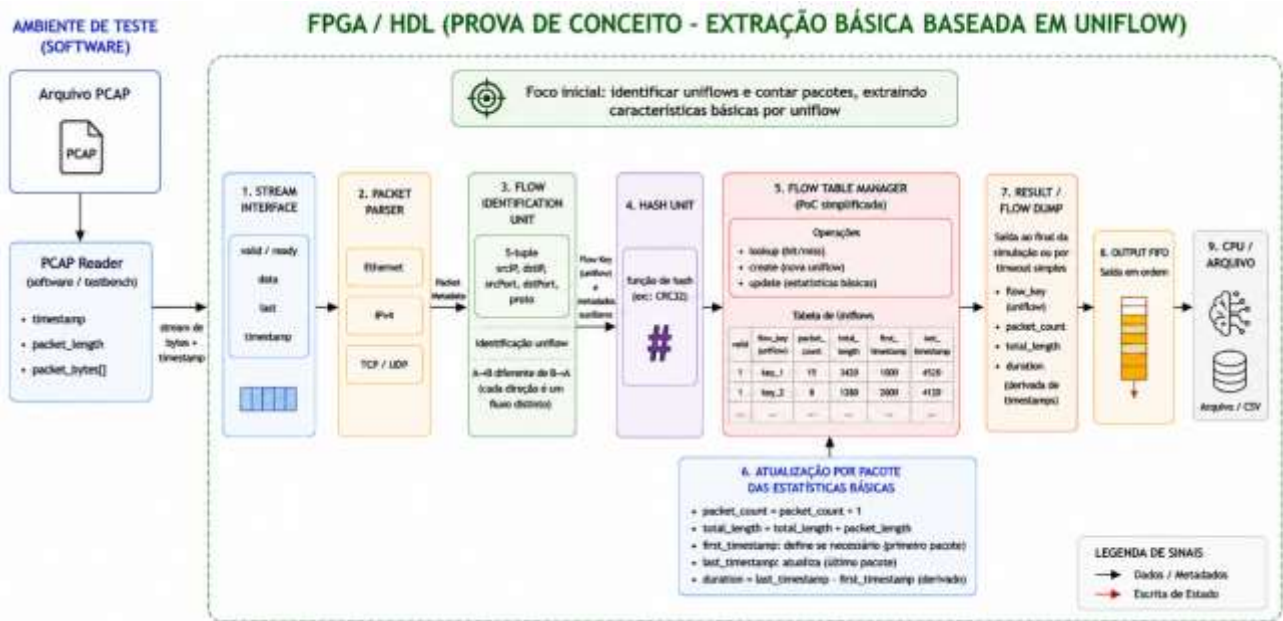


Visão geral da arquitetura do extrator de features baseado em fluxo:



Nesta primeira etapa, o objetivo da prova de conceito é receber pacotes de um arquivo PCAP, identificar a qual uniflow cada pacote pertence e contabilizar o número de pacotes, o comprimento total dos pacotes e a duração de cada fluxo.

Assim, a primeira versão do sistema deverá produzir, para cada uniflow identificada, basicamente:

- flow_key
- packet_count
- total_length
- duration

Casos mais complexos, como fluxos bidirecionais, cálculo de médias, variância, IAT, tratamento elaborado de colisões, timeout, FIN/RST e um conjunto completo de features, ficam como extensões futuras.

1. Stream interface (pcap)

O objetivo da interface de entrada é transformar os pacotes armazenados no arquivo PCAP em um **stream de palavras compatível com a entrada do Packet Parser**.

Para cada pacote, os principais sinais são:

- data: bytes do quadro Ethernet;
- timestamp: instante de chegada do pacote;
- valid: indica que a fonte está apresentando dados válidos;

- ready: indica que o parser pode receber novos dados;
- sop: indica a primeira palavra do pacote;
- eop: indica a última palavra do pacote;
- keep: indica quais bytes da última palavra são válidos.

Como nesta etapa o projeto será desenvolvido apenas em simulação, o arquivo PCAP será lido por software. Existem duas possibilidades:

1. **Offline:** um programa em Python lê o PCAP, separa os pacotes e gera previamente um arquivo contendo os valores que serão utilizados pelo testbench.
2. **Online:** uma ferramenta de integração, como o Cocotb, utiliza Python diretamente durante a simulação para ler o PCAP e estimular os sinais do módulo HDL.

Para a primeira prova de conceito, a abordagem **offline é a alternativa mais simples**, pois reduz a complexidade do ambiente de simulação. A integração com Cocotb pode ser feita posteriormente.

O Python não precisa transmitir eletricamente os pacotes para a FPGA durante a simulação. Sua função é apenas interpretar o arquivo PCAP e fornecer ao testbench os bytes e o timestamp de cada pacote.

Por exemplo, considere o seguinte pacote:

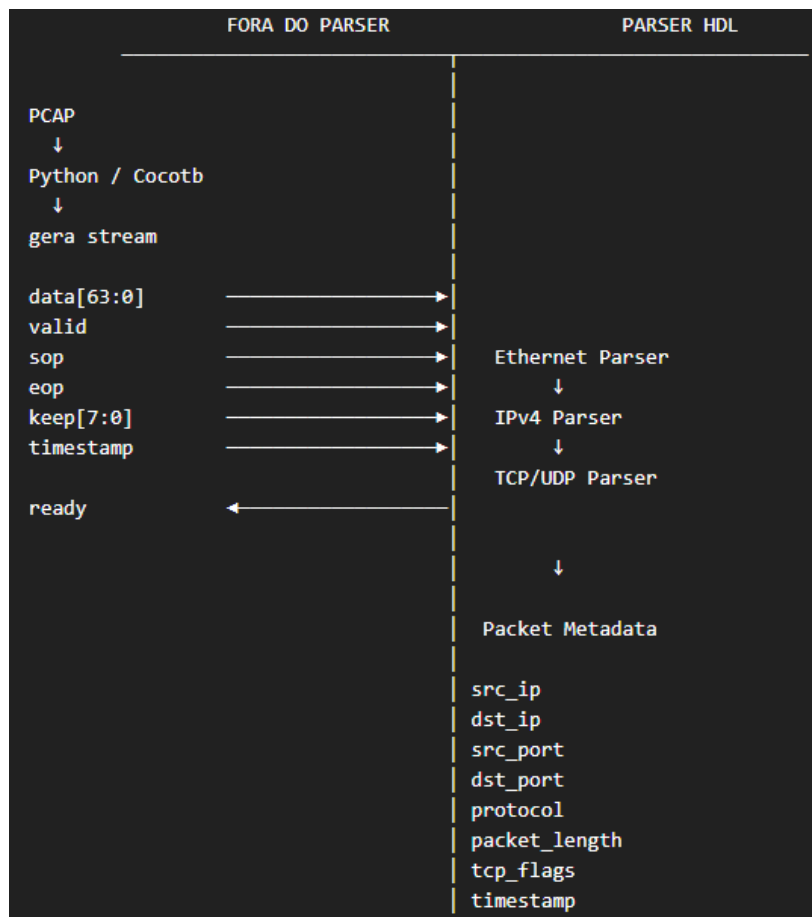
- timestamp = 1.234567 s
- length = 20 bytes
- packet = AA BB CC DD EE FF 11 22 33 44 55 66 08 00 45 00 12 34 56 78

Supondo uma interface de 64 bits (8 bytes) por ciclo, o testbench poderia gerar:

ciclo	data	sop	eop	valid	keep
0	AABBCCDDEEFF1122	1	0	1	FF
1	3344556608004500	0	0	1	FF
2	1234567800000000	0	1	1	F0

O próprio leitor do PCAP sabe onde cada pacote começa e termina. Portanto, o testbench pode gerar sop, eop e keep a partir do comprimento de cada pacote.

Em uma eventual implementação física, uma interface de rede ou MAC deverá gerar sinais equivalentes. Dessa forma, a arquitetura a partir do Packet Parser pode permanecer praticamente inalterada.



2. Packet Parser

O objetivo do Packet Parser é receber o quadro Ethernet em formato de stream e extrair apenas os campos necessários para identificar a uniflow.

Nesta primeira versão, serão considerados apenas: Ethernet, IPv4 e TCP / UDP. Casos como IPv6, VLAN, fragmentação IP, cabeçalhos opcionais e protocolos encapsulados ficam inicialmente fora do escopo.

Antes do parser, existe apenas um conjunto de bytes sem interpretação. Após o parser, esses bytes passam a ser representados por metadados estruturados.

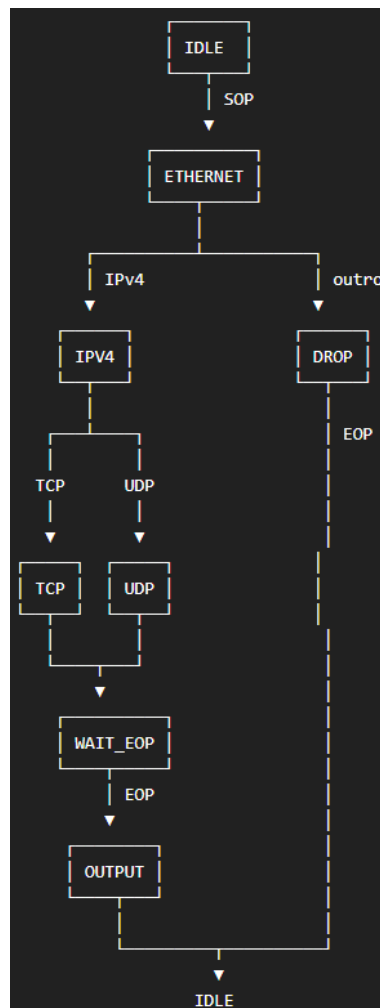
Para a prova de conceito, a saída pode ser reduzida a:

- src_ip
- dst_ip
- src_port
- dst_port
- protocol
- packet_length
- timestamp

Os primeiros cinco campos serão utilizados para identificar a uniflow.

O packet_length e o timestamp são necessários para as features da primeira prova de conceito: total_length e duration.

Uma FSM pode ser utilizada para controlar o parser. Cada estado indica qual parte do pacote está sendo interpretada naquele momento. O parser não precisa armazenar o pacote inteiro. Ele pode interpretar os dados à medida que as palavras chegam e guardar apenas os campos de interesse.



3. Flow identification

O objetivo da Flow Identification Unit é receber os metadados produzidos pelo Packet Parser e construir a **chave da uniflow**.

Nesta primeira versão, não haverá canonicalização bidirecional. A chave será diretamente a 5-tuple: **flow_key = {src_ip, dst_ip, src_port, dst_port, protocol}**

Assim, cada direção da comunicação será considerada um fluxo diferente. Por exemplo:

- src_ip = 192.168.1.10
- dst_ip = 8.8.8.8
- src_port = 52100
- dst_port = 443
- protocol = TCP

Fluxo de ida: 192.168.1.10:52100 -> 8.8.8.8:443 (TCP)

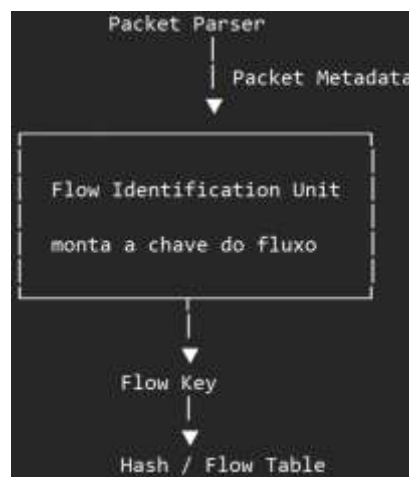
- chave: {192.168.1.10, 8.8.8.8, 52100, 443, TCP}

Fluxo de volta: 8.8.8.8:443 -> 192.168.1.10:52100 (TCP)

- chave: {8.8.8.8, 192.168.1.10, 443, 52100, TCP}

A Flow Identification Unit apenas agrupa os campos necessários e encaminha: **flow_key** + **metadados auxiliares** para a Hash Unit.

Na primeira PoC, os metadados auxiliares podem ser simplesmente: timestamp e packet_length.



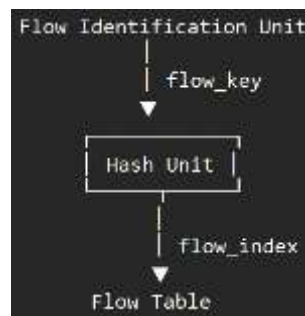
4. Hash unit

O objetivo da Hash Unit é receber a flow_key da uniflow e gerar um valor menor que possa ser utilizado para acessar a Flow Table.

A entrada é: {src_ip, dst_ip, src_port, dst_port, protocol}

A saída é: hash -> index

O objetivo inicial não é avaliar qual algoritmo de hash possui a melhor distribuição, mas apenas demonstrar que a chave de uma determinada uniflow produz de forma determinística o mesmo índice.



5. Flow table

A Flow Table é responsável por **manter o estado das uniflows** identificadas durante o processamento do PCAP.

Na primeira prova de conceito, cada entrada da tabela pode armazenar:

- **valid:** indica se a posição está ocupada;
- **flow_key:** chave da uniflow, formada pela 5-tuple;
- **packet_count:** número de pacotes observados na uniflow;
- **total_length:** soma dos comprimentos dos pacotes da uniflow;
- **first_timestamp:** timestamp do primeiro pacote;
- **last_timestamp:** timestamp do pacote mais recente.

valid	flow_key	packet_count	total_length	first_ts	last_ts
1	key_1	15	8200	1000	4500
1	key_2	8	3700	1200	3100
0	-	-	-	-	-

Ao receber um pacote, a tabela verifica a posição correspondente.

- **Hit:** se a entrada é válida e contém a mesma *flow_key*, ocorre um hit. Nesse caso, o estado atual da uniflow é recuperado e encaminhado para o módulo de atualização das estatísticas.
- **Miss:** se a entrada ainda não estiver sendo utilizada, ocorre um miss e uma nova uniflow é criada. A entrada é inicializada com os dados do primeiro pacote.

Colisão de hash: Para a primeira prova de conceito, não é necessário implementar uma política sofisticada de tratamento de colisões. A tabela deve armazenar a *flow_key* completa justamente para permitir verificar se a posição encontrada pertence realmente ao fluxo procurado. Se a posição indicada pelo hash estiver válida, mas possuir outra *flow_key*, uma colisão é detectada. Inicialmente, esse caso pode apenas gerar uma indicação de erro, utilizando-se PCAPs controlados para evitar colisões durante os primeiros testes.

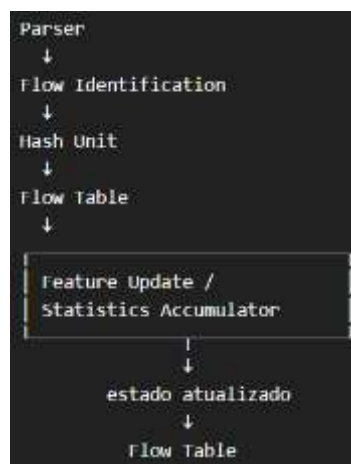
6. Feature/statistics update

Na primeira prova de conceito, o módulo **Feature / Statistics Update** será responsável por atualizar três informações básicas associadas à uniflow: número de pacotes, comprimento total e timestamp do pacote mais recente.

Quando ocorre um *hit*, o módulo recebe o estado atual da uniflow e os metadados do novo pacote e realiza:

- **packet_count_new = packet_count + 1**
- **total_length_new = total_length + packet_length**
- **last_timestamp_new = timestamp**

A operação segue o padrão **Read-Modify-Write**:



Depois que essa arquitetura estiver funcionando, o mesmo bloco pode ser expandido incrementalmente para: **min/max packet_length, IAT e outras estatísticas**.

7. Flow finalizer

Na arquitetura completa, o **Flow Finalizer** seria responsável por transformar o estado acumulado de uma uniflow em um vetor de features, realizando cálculos derivados como:

- média;
- variância;
- desvio padrão;
- taxas;
- duration.

Entretanto, esses cálculos mais complexos não serão necessários na primeira prova de conceito.

Inicialmente, **apenas a feature duration será calculada pelo Flow Finalizer**, a partir dos timestamps armazenados na Flow Table: **duration = last_timestamp - first_timestamp**.

As features **packet_count** e **total_length** já são acumuladas durante o processamento dos pacotes pelo módulo de Feature/Statistics Update e armazenadas diretamente na Flow Table. Portanto, ao final da simulação, seus valores podem ser obtidos por meio de um **simple dump das entradas válidas da tabela**.

8. Primeira evolução: biflow e ampliação das features

Após a validação da prova de conceito baseada em **uniflows**, havendo tempo hábil, pretende-se incrementar o projeto com algumas funcionalidades:

- **Comunicação biflow:** consiste em agrupar os pacotes dos dois sentidos de uma mesma comunicação em uma única entrada da Flow Table. Para isso, a Flow Identification Unit deverá realizar a **canonicalização da 5-tuple**, garantindo que $A \rightarrow B$ e $B \rightarrow A$ produzam a mesma **flow_key**, além de indicar a direção do pacote como **FWD** ou **BWD**. A partir disso, poderão ser mantidos contadores separados por direção, como **fwd_packet_count**, **bwd_packet_count**, **fwd_total_length** e **bwd_total_length**.
- **Features packets/s e bytes/s:** poderão ser calculadas no Flow Finalizer a partir dos valores já disponíveis na arquitetura: **packet_count**, **total_length** e **duration**. A principal complexidade adicional está na implementação da **divisão**, que pode ser realizada de forma sequencial durante a finalização da unifiow.
- **Features de cabeçalho:** também poderão ser adicionadas informações extraídas diretamente dos cabeçalhos dos pacotes. No caso do TCP, podem ser mantidos contadores para as flags **SYN**, **ACK**, **FIN**, **RST**, **PSH** e **URG**, indicando quantas vezes cada flag apareceu durante a unifiow. Outras informações simples, como tamanho do cabeçalho, protocolo ou campos específicos do IPv4/TCP, também podem ser incorporadas. Como essas features dependem principalmente de **extração de bits e incrementos de contadores**, sua implementação tende a ser menos complexa do que features que exigem operações matemáticas, como médias ou variâncias.